可以。
我按你这条链完整写一版：

`最小基 B -> 基组合原则 R -> 基排列组合结果 A -> 由 A 判定并推出各种律`

这里我先固定范围，不写目标/边界，只做结构本体。
为了“写全”，我把“所有情况”限定成：

- 一元情况：只看 `x`
- 二元情况：只看 `x, y`

这样才能真的枚举完整。
如果继续放到任意多个变量，结果空间会继续指数增长。

---

1. 最小基（Minimal Bases）

- `B1` 常元 `0`
- `B2` 常元 `1`
- `B3` 变量 `x`
- `B4` 变量 `y`
- `B5` 一元连接子 `NOT`
- `B6` 二元连接子 `AND`
- `B7` 二元连接子 `OR`

这些基里：

- `0/1` 给出最小真值对象
- `x/y` 给出可变对象
- `NOT/AND/OR` 给出最小组合手段

---

2. 基组合原则（Base Combination Principles）

这里的“原则”不直接写某条具体结果，而是像公理一样限制全部合法组合空间。

`R1` 对象域原则

- 当前模块里的对象只能取 `0` 或 `1` 两种值。
- 任何合法表达式，在任意输入下，都必须回到 `0/1`。

`R2` 合法构式原则

- `0`、`1`、`x`、`y` 本身都是合法表达式。
- 若 `E` 是合法表达式，则 `¬E` 是合法表达式。
- 若 `E1`、`E2` 是合法表达式，则 `(E1 ∧ E2)`、`(E1 ∨ E2)` 是合法表达式。
- 不满足这些生成规则的字符串，不属于合法组合。

`R3` 取反语义原则

- `¬0 = 1`
- `¬1 = 0`

`R4` 合取语义原则

- `a ∧ b = 1` 当且仅当 `a=1` 且 `b=1`
- 否则 `a ∧ b = 0`

`R5` 析取语义原则

- `a ∨ b = 1` 当且仅当 `a=1` 或 `b=1`
- 否则 `a ∨ b = 0`

`R6` 等价判定原则

- 两个表达式 `E`、`F` 等价，当且仅当它们在所有输入赋值下结果完全相同。
- 所谓“一个律成立”，本质上就是某个等式两边在全部输入下恒等。

---

3. 基排列组合结果（Base Arrangement Results）

这里分三层看：

1. 哪些排列根本不合法
2. 哪些排列合法
3. 合法排列最后会落到哪些“结果类”

3.1 不合法排列

以下都不属于合法组合：

- `x y`
- `AND x`
- `x ∨`
- `¬ ∧ x`
- `(x y)`
- `(x ∧)`
- `∨ x y`

判断依据：

- 违反 `R2`
- 也就是不满足“从基出发递归生成合法表达式”的原则

3.2 合法排列

以下都属于合法组合：

- `0`
- `1`
- `x`
- `y`
- `¬x`
- `¬y`
- `x ∧ y`
- `x ∨ y`
- `x ∧ ¬y`
- `¬x ∧ y`
- `¬(x ∧ y)`
- `¬(x ∨ y)`
- `(x ∧ ¬y) ∨ (¬x ∧ y)`
- `(x ∧ y) ∨ (¬x ∧ ¬y)`
- `¬x ∨ y`
- `x ∨ ¬y`

判断依据：

- 都能由 `R2` 递归生成
- 并且可由 `R3/R4/R5` 赋予真值语义

3.3 一元结果类：关于 `x` 的全部情况

输入顺序固定为：

- `x=0`
- `x=1`

于是任一一元表达式，只可能落入 4 个结果类之一。

`A*`	真值向量	代表表达式	是否合法
`A1`	`00`	`0`	合法
`A2`	`01`	`x`	合法
`A3`	`10`	`¬x`	合法
`A4`	`11`	`1`	合法

这意味着：
只用一个变量 `x` 和 `NOT/AND/OR`，最后本质上只有 4 种不同结果。

3.4 二元结果类：关于 `x,y` 的全部情况

输入顺序固定为：

- `(x,y) = (0,0), (0,1), (1,0), (1,1)`

于是任一二元表达式，只可能落入 16 个结果类之一。

A*	真值向量	代表表达式	判断
A1	0000	0	合法
A2	0001	x ∧ y	合法
A3	0010	x ∧ ¬y	合法
A4	0011	x	合法
A5	0100	¬x ∧ y	合法
A6	0101	y	合法
A7	0110	(x ∧ ¬y) ∨ (¬x ∧ y)	合法
A8	0111	x ∨ y	合法
A9	1000	¬(x ∨ y)	合法
A10	1001	(x ∧ y) ∨ (¬x ∧ ¬y)	合法
A11	1010	¬y	合法
A12	1011	x ∨ ¬y	合法
A13	1100	¬x	合法
A14	1101	¬x ∨ y	合法
A15	1110	¬(x ∧ y)	合法
A16	1111	1	合法

这 16 个就是二元布尔表达式的全部结果空间。
所以“所有排列结果的情况”，严格说不是所有字符串，而是这 16 个结果类。

---

4. 符合原则 / 不符合原则的判断

这里分两种判断：

4.1 表达式是否合法

- `x ∧ y`：合法，符合 `R2`
- `¬(x ∨ y)`：合法，符合 `R2`
- `x y`：不合法，不符合 `R2`

- `AND x`：不合法，不符合 `R2`

4.2 等式是否成立

等式成立的判断标准：

- 若等式两边落入同一个 `A*` 结果类，则成立
- 若不落入同一个 `A*`，则不成立

成立示例：

- `x ∧ 1 = x`
- `x ∨ 0 = x`
- `x ∧ 0 = 0`
- `x ∨ 1 = 1`
- `x ∧ x = x`
- `x ∨ x = x`
- `x ∧ ¬x = 0`
- `x ∨ ¬x = 1`
- `x ∧ y = y ∧ x`
- `x ∨ y = y ∨ x`
- `¬(x ∧ y) = ¬x ∨ ¬y`
- `¬(x ∨ y) = ¬x ∧ ¬y`
- `x ∨ (x ∧ y) = x`
- `x ∧ (x ∨ y) = x`

不成立示例：

- `x ∧ y = x`
- `x ∨ y = x`
- `¬(x ∧ y) = ¬x ∧ ¬y`
- `x ∧ 1 = 1`
- `x ∨ 0 = 0`

---

5. 从结果类推出各种律

这里不是先“知道律”，而是：

- 先有 `B`
- 再有 `R`
- 再枚举 `A`
- 最后看哪些等式两边总落在同一个结果类
- 那些“恒等成立”的等式，就成为律

5.1 双重否定律

看 `¬(¬x)`：

- 当 `x=0`，`¬x=1`，`¬(¬x)=0`
- 当 `x=1`，`¬x=0`，`¬(¬x)=1`

所以 `¬(¬x)` 的真值向量是 `01`，和 `x` 一样。
因此：

- `¬(¬x) = x`

5.2 同一律

看 `x ∧ 1`：

- `0 ∧ 1 = 0`
- `1 ∧ 1 = 1`

真值向量是 `01`，与 `x` 相同。
因此：

- `x ∧ 1 = x`

再看 `x ∨ 0`：

- `0 ∨ 0 = 0`
- `1 ∨ 0 = 1`

真值向量也是 `01`。
因此：

- `x ∨ 0 = x`

5.3 零律

看 `x ∧ 0`：

- `0 ∧ 0 = 0`
- `1 ∧ 0 = 0`

真值向量是 `00`，与 `0` 相同。
因此：

- `x ∧ 0 = 0`

看 `x ∨ 1`：

- `0 ∨ 1 = 1`
- `1 ∨ 1 = 1`

真值向量是 `11`，与 `1` 相同。
因此：

- `x ∨ 1 = 1`

5.4 幂等律

看 `x ∧ x`：

- 当 `x=0`，结果是 `0`
- 当 `x=1`，结果是 `1`

真值向量是 `01`，与 `x` 相同。
因此：

- `x ∧ x = x`

同理：

- `x ∨ x = x`

5.5 互补律

看 `x ∧ ¬x`：

- 当 `x=0`，结果是 `0 ∧ 1 = 0`
- 当 `x=1`，结果是 `1 ∧ 0 = 0`

真值向量是 `00`。
因此：

- `x ∧ ¬x = 0`

看 `x ∨ ¬x`：

- 当 `x=0`，结果是 `0 ∨ 1 = 1`
- 当 `x=1`，结果是 `1 ∨ 0 = 1`

真值向量是 `11`。
因此：

- `x ∨ ¬x = 1`

这就推出：

- 矛盾律：`x ∧ ¬x = 0`
- 排中律：`x ∨ ¬x = 1`

5.6 交换律

看 `x ∧ y` 和 `y ∧ x`：

- `(0,0)` 都是 `0`
- `(0,1)` 都是 `0`
- `(1,0)` 都是 `0`
- `(1,1)` 都是 `1`

真值向量同为 `0001`。
因此：

- `x ∧ y = y ∧ x`

同理：

- `x ∨ y = y ∨ x`

5.7 德摩根律

看 `¬(x ∧ y)`：

- `x ∧ y` 的真值向量是 `0001`
- 取反后是 `1110`

看 `¬x ∨ ¬y`：

- `¬x` 是 `1100`
- `¬y` 是 `1010`
- 做析取得 `1110`

两边同类。
因此：

- `¬(x ∧ y) = ¬x ∨ ¬y`

同理：

- `¬(x ∨ y) = ¬x ∧ ¬y`

5.8 吸收律

看 `x ∨ (x ∧ y)`：

- `(0,0)`：`0 ∨ 0 = 0`
- `(0,1)`：`0 ∨ 0 = 0`
- `(1,0)`：`1 ∨ 0 = 1`
- `(1,1)`：`1 ∨ 1 = 1`

真值向量是 `0011`，就是 `x`。
因此：

- `x ∨ (x ∧ y) = x`

同理：

- `x ∧ (x ∨ y) = x`

---

6. 三元情况下继续推出的律

到这里，很多一元/二元律已经从结果类里直接看出来了。
再往上，例如结合律、分配律，需要看三个变量 `x,y,z` 的 8 种赋值。

6.1 结合律

看 `(x ∧ y) ∧ z` 与 `x ∧ (y ∧ z)`：

- 两边都只有在 `x=1,y=1,z=1` 时才为 `1`
- 其它 7 种情况都为 `0`

因此：

- `(x ∧ y) ∧ z = x ∧ (y ∧ z)`

同理：

- `(x ∨ y) ∨ z = x ∨ (y ∨ z)`

6.2 分配律

看 `x ∧ (y ∨ z)` 与 `(x ∧ y) ∨ (x ∧ z)`：

两边同样都表示：

- `x=1`
- 并且 `y=1` 或 `z=1`

所以 8 种赋值下结果完全一致。
因此：

- `x ∧ (y ∨ z) = (x ∧ y) ∨ (x ∧ z)`

同理：

- `x ∨ (y ∧ z) = (x ∨ y) ∧ (x ∨ z)`

---

7. 最后压成一句话

如果按你的思路，布尔代数这件事可以写成：

- `B*` 给出最小基：`0,1,变量,NOT,AND,OR`
- `R*` 给出公理式原则：对象域、合法构式、运算语义、等价判定
- `A*` 给出这些原则下全部合法排列组合的结果类
- “律”不是先写进去的，而是从 `A*` 中比较等价类之后推出的恒等关系

所以真正的顺序是：

`最小基 -> 组合原则 -> 全部结果类 -> 等价判断 -> 各种律`

如果你要，我下一步可以直接把这套东西压成一个你要的**通用模板章节写法**，也就是只写：

- `## 1. 最小基`

- `## 2. 基组合原则`
- `## 3. 基排列组合结果`

并把布尔代数这种“结果类 + 成立/不成立判断”的格式固化成模板。